

PHP pliki cd., CMS, XSS

Wykłady 13

Bartosz Lauks

19 maja 2026



- 1 Udostępnianie hostowanych plików i dostępy
- 2 CMS
- 3 XSS

Cele wykładu

- Implementacja systemu ról i uprawnień
- Ograniczanie dostępu do zasobów
- Udostępnianie hostowanych plików.
- Tworzenie prostego CMS
- Zabezpieczenie aplikacji przed XSS

① Udostępnianie hostowanych plików i dostępy

② CMS

③ XSS

Udostępnianie hostowanych plików i dostępy

Rozwinięciem systemu tworzenia postów

- W poprzednim wykładzie omówiliśmy jak operować na plikach przy pomocy PHP oraz w jaki sposób zapisywać pliki przesłane przez klienta po stronie serwera.
- Wykorzystując tą wiedzę w praktyce rozbudujemy system postów i komentarz o dodatkowo funkcjonalność obejmującą:
 - Dodawanie postów (obecnie każdy użytkownik może dodawać jedynie komentarze)
 - Każdy post będzie dodatkowo powiązany z zdjęciem.
 - Posty będzie mógł dodawać tylko użytkownik z rolą **admin**
 - Post oprócz tytułu oraz opisu będzie zawierał też **zdjęcie**.

Udostępnianie hostowanych plików i dostępny

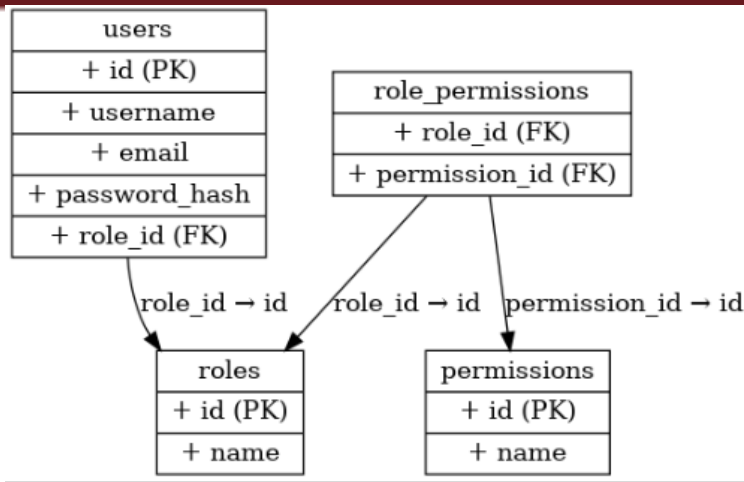
Role i uprawnienia w serwisach WWW

- W celu zapewnienia kontroli nad dostępem do zasobów aplikacji oraz zabezpieczenia krytycznych operacji wykonywanych na serwerze, konieczne jest wdrożenie odpowiedniego mechanizmu zarządzania uprawnieniami użytkowników.
- Mechanizm ten powinien umożliwiać definiowanie, kto i w jakim zakresie może korzystać z funkcji systemu.
- Jednymi z najpopularniejszych modeli ról i uprawnień są:
 - **RBAC (Role-Based Access Control)** - klasyczny model, w którym każda rola ma przypisane uprawnienia.
 - **ACL (Access Control List)** – lista kontroli dostępu przypisana bezpośrednio do konkretnego zasobu.
 - **ABAC (Attribute-Based Access Control)** - bardziej elastyczny model, oparty na atrybutach użytkownika, zasobów i kontekstu.

Udostępnianie hostowanych plików i dostępny

RBAC (Role-Based Access Control)

- Najpopularniejszy, uprawnienia przypisywane są rolom, a nie bezpośrednio użytkownikom. Przykład:
 - **Admin** - create_post, delete_post, ban_use
 - **Editor** - create_post, edit_post
 - **Viewer** - read_post
- Dane o roli użytkownika mogą być przechowywane w bazie danych.
- Przykładowo użytkownik z rolą **Editor** może tworzyć i edytować posty, ale nie może ich usuwać ani banować użytkowników.



Rysunek 1: Dogram schematu bazy danych do przechowywania dostępu użytkownika

Udostępnianie hostowanych plików i dostępy

ACL (Access Control List)

- To odwrotność RBAC w tym podejściu, to każdy **zasób** ma listę, kto i co może z nim zrobić. Przykład:
- Raporty PDF:
 - **user** : ala:
 - **permissions** : read
 - **permissions** : write
 - **user** : ola:
 - **permissions** : read
- Ala może czytać i edytować plik. Ola może tylko czytać.

Udostępnianie hostowanych plików i dostępy

ABAC (Attribute-Based Access Control)

- Dostęp opiera się na atrybutach użytkownika, zasobu i środowiska.
- Jest trudniejszy do realizacji lecz daje większe możliwości elastycznego przydzielania dostępu. Często stosowany w systemach rządowych oraz korporacyjnych.
- Tak jak w przypadku RBAC polega na nadanie pewnych ról użytkownikowi, ale oprócz tego role nadawane są także zasobom. Przykład:
- **Użytkownik:**
 - **username** : kasia
 - **department** : HR
 - **security_clearance** : high

Udostępnianie hostowanych plików i dostępy

ABAC (Attribute-Based Access Control cd.)

- **Zasób:**
 - **document** : employee_records.pdf,
 - **required_clearance** : high,
 - **department** : HR
- Użytkownik może uzyskać dostęp do dokumentu, jeśli:
 - **clearance użytkownika** $\geq$ **clearance zasobu**
 - **department użytkownika** $=$ **department zasobu**

Udostępnianie hostowanych plików i dostępy

Podstawowa wersja nadawania roli admin

- Aby maksymalnie uprościć zabezpieczenie funkcjonalności dodawania postów tylko dla administratora, w tablicy **user** dodano nowe pole:
 - **isAdmin** - (`tinyint(1)`)
- Na podstawie tego pola serwer ustawi w zmiennej sesyjnej `$_SESSION['isAdmin']` wartość `true`, jeśli użytkownik ma uprawnienia administratora, i będzie na jej podstawie weryfikować dostęp do zasobów.

<https://localhost/lecture13/loginPDO.php>

Udostępnianie hostowanych plików i dostępy

Podstawowa wersja nadawania roli admin cd.

- W pliku `permission_check.php` została zaimplementowana logika sprawdzająca, czy użytkownik ma dostęp do określonego zasobu.
- Skrypt został wydzielony do osobnego pliku, aby można go było dołączać za pomocą `require_once` w miejscach, gdzie dostęp powinien być ograniczony tylko do administratora.
- Tego samego skryptu można również użyć do kontrolowania widoczności elementów interfejsu, takich jak link do dodawania nowych postów tylko dla administratorów.

https://localhost/lecture13/permission_check.php

Udostępnianie hostowanych plików i dostępy

Przechowywanie zdjęć dla postów

- Przesłane przez administratora zdjęcie podczas tworzenia posta zostanie poddane operacjom omówionym na poprzednim wykładzie:
 - **walidacja** - sprawdzenie, czy plik jest zdjęciem (rozszerzenie, rozmiar, typ MIME itp.),
 - **anonimizacja** - nadanie unikalnej nazwy w celu uniknięcia nadpisania istniejących plików,
 - **zapis** - umieszczenie pliku w podfolderze [uploads/](#).

Udostępnianie hostowanych plików i dostępy

Przechowywanie zdjęć dla postów

- Nazwa pliku zostanie zapisana w nowej kolumnie **image** w tabeli **posts**.
- W ten sposób zdjęcie zostanie powiązane z postem. Gdy klient pobierze dane postów, otrzyma również ścieżkę do pliku graficznego na serwerze.
- Przeglądarka użytkownika automatycznie pobierze zdjęcie na podstawie tej ścieżki i wyświetli je w interfejsie.
- Ważne, aby dostęp do katalogu **uploads/** nie był blokowany.
- Upewnij się że nie blokujesz dostępu do zasobów przy pomocy **.htaccess** lub systemowo (**ls -l**)

<https://localhost/lecture13/createNewPost.php>

<https://localhost/lecture13/postPagination.php>

Definicja CMS

- **CMS (ang. Content Management System)** to System Zarządzania Treścią - aplikacja lub zestaw narzędzi służących do tworzenia, edytowania, publikowania i zarządzania treścią stron internetowych bez konieczności znajomości języków programowania.
- System CMS oddziela warstwę prezentacji (czyli wygląd strony) od warstwy treści (czyli zawartości: tekstów, zdjęć, filmów itp.), umożliwiając osobom nietechnicznym samodzielne zarządzanie witryną.

Główne cechy CMS

- **Interfejs użytkownika (GUI)** - zazwyczaj dostęp przez przeglądarkę, umożliwia intuicyjne zarządzanie treścią (tzw. WYSIWYG - „What You See Is What You Get”).
- **Panel administracyjny** - umożliwia dodawanie stron, edycję artykułów, zarządzanie menu, użytkownikami, komentarzami itp.
- **Szablony i motywy graficzne** - pozwalają łatwo zmieniać wygląd strony bez ingerencji w kod.
- **Modułowość i rozszerzalność** - poprzez dodatki (pluginy, moduły) można rozszerzyć funkcjonalność strony.
- **System uprawnień** - zarządzanie dostępem użytkowników (np. redaktor, administrator, gość).
- **Wersjonowanie treści** - możliwość cofnięcia się do poprzednich wersji

Dlaczego warto używać CMS?

- **Szybkie wdrożenie strony** - gotowe szablony i funkcje.
- **Łatwość obsługi** - nawet osoby bez wiedzy technicznej mogą zarządzać stroną.
- **Zwiększenie efektywności pracy zespołowej** - wiele osób może współpracować przy tworzeniu strony.
- **Elastyczność** - możliwość dostosowania CMS do różnych typów serwisów (blog, sklep, portal firmowy).
- **Oszczędność czasu i kosztów** - mniejsze potrzeby wsparcia technicznego.

Podział CMS-ów

- Ze względu na dostępność kodu:
 - **Open Source (otwarte oprogramowanie)** - np. WordPress, Joomla!, Drupal.
 - **Komercyjne (płatne)** - np. Kentico, Sitecore, Adobe Experience Manager.
- Ze względu na przeznaczenie:
 - **CMS ogólnego przeznaczenia** - WordPress, Joomla.
 - **CMS do e-commerce** - PrestaShop, Magento, WooCommerce.
 - **CMS do blogowania** - Ghost, Blogger.
 - **CMS do portali społecznościowych / intranetów** - eXo Platform, Liferay.

Architektura CMS-a (ogólnie)

- **Bazy danych** - przechowuje treści, użytkowników, konfigurację.
- **Warstwy aplikacji** (backend) - logika biznesowa, obsługa żądań użytkownika.
- **Warstwy prezentacji** (frontend) - generowanie stron widocznych dla odwiedzających.
- **Panelu administracyjnego** - dostępnego zwykle po zalogowaniu.

Implementacji prostego CMS

- Na potrzeby wykładu stworzymy prosty system CMS.
- Prawie każdy CMS przechowuje zawartość wyświetlanych stron w bazie danych.
- Dlatego kolejnym krokiem będzie stworzenie tabeli **cms**, która będzie posiadała kolumny takie jak:
 - **id (int(11))** - klucz główny,
 - **content (text)** - pole, w którym przechowywany będzie kod **HTML** wyświetlany użytkownikom,
 - **hide (tinyint(1)) Default=1** - pole odpowiadające za ukrywanie treści na stronie,
 - **orders (int(11)) Default=1** - pole służące do manipulowania kolejnością treści na stronie,
 - **created_at (timestamp)** - pole do przechowywania daty utworzenia zasobu; wykorzystywane także do określania kolejności wyświetlania treści na stronie.

Implementacji prostego CMS cd.

- Zazwyczaj systemy CMS posiadają **panel administracyjny**, w którym administratorzy dodają, edytują oraz usuwają treści na stronie.
- W naszym przypadku, aby maksymalnie uprościć aplikację, wykorzystamy **phpMyAdmin**.
- Logując się do niego, możemy ręcznie tworzyć nowe rekordy w bazie danych i umieszczać tam fragmenty strony.
- Lub manipulując polem **hide** i **orders** ukrywać lub przestawiać szysk sekcji storny.

Implementacji prostego CMS cd.

- Kolejnym krokiem jest utworzenie podstrony [blog.php](#).
- To właśnie ona będzie pełniła rolę strony CMS.
- Zawiera szablon widoku (style, nagłówek, stopkę itp.).
- Jej głównym elementem jest kontener `<div class="container">`; to w nim skrypt **PHP** będzie pobierał z bazy danych kod sekcji (**HTML**, **CSS**, **JS...**) i publikował go na stronie.

<https://localhost/lecture13/blog.php>

① Udostępnianie hostowanych plików i dostępy

② CMS

③ XSS

Atak XSS (Cross-Site Scripting)

- **Ataki XSS (Cross-Site Scripting)** to jedna z najczęstszych i najbardziej niebezpiecznych luk bezpieczeństwa w aplikacjach webowych.
- Polegają na wstrzyknięciu złośliwego kodu **JavaScript** (lub innego kodu wykonywalnego po stronie klienta) do treści strony, która następnie zostaje wyświetlona innym użytkownikom.
- Celem ataków XSS jest zwykle kradzież danych, przejęcie sesji, wyświetlanie fałszywych treści lub przekierowywanie użytkowników.

XSS

Na czym polega atak XSS?

- XSS wykorzystuje to, że aplikacja webowa niewłaściwie przetwarza dane wejściowe użytkownika i bez odpowiedniej walidacji lub oczyszczania (**sanityzacji**) wyświetla je innym użytkownikom.
- W efekcie, jeśli atakujący wprowadzi np. kod JavaScript jako dane wejściowe (np. w formularzu), ten kod może zostać uruchomiony w przeglądarce innego użytkownika.

XSS

Przykład ataku

- Załóżmy że nasza strona powinna wyświetlić nam tekst powitalny np.
`<p>Witaj, Janek!</p>`
- Ale jeśli dane wejściowe są takie:
`<script>alert('Hacked!');</script>`
- i aplikacja je wyświetli w ten sposób: `<p>Witaj,`
`<script>alert('Hacked!');</script>!</p>`
- to zostanie wykonany złośliwy kod JavaScript.

XSS

Typy ataku

- **Stored XSS (trwały)** - Dane wprowadzone przez atakującego są trwale zapisane w bazie danych lub innym trwałym magazynie (np. w komentarzach, profilach, postach). Gdy inny użytkownik otworzy stronę z tą treścią, złośliwy skrypt się uruchamia.
- **Reflected XSS (odbijany)** - Dane wprowadzone przez użytkownika nie są zapisywane, ale są natychmiast zwracane w odpowiedzi. Najczęściej dotyczy to parametrów w adresie URL.

https://example.com/search?q=<script>stealCookies()</script>

XSS

Skutki ataków XSS

- Kradzież ciasteczek sesyjnych
- Przejęcie konta użytkownika
- Podmiana treści strony (np. fałszywe formularze logowania)
- Przekierowanie na złośliwe strony
- Keyloggery i inne szpiegowskie skrypty
- Ataki typu phishing (podszywanie się pod znaną stronę)

XSS

Jak zapobiegać atakom XSS?

- **Filtrowanie i walidacja danych wejściowych** - Nie powinno się ufać żadnym danym od użytkownika. Dane należy sprawdzać, czy są zgodne z oczekiwanym formatem (np. e-mail, liczba).
- **Escapowanie danych wyjściowych (output encoding)** - Zawsze należy zamieniać znaki specjalne HTML (<, >, &, " itd.) przed wyświetleniem ich na stronie:
 - " → "
 - ' → '
 - & → &
- w PHP umożliwia to funkcja **htmlspecialchars()** służy do zamiany specjalnych znaków w tekście na ich bezpieczne odpowiedniki w HTML. Dzięki temu zapobiega m.in. atakom XSS.

XSS

Jak zapobiegać atakom XSS?

- Niestety, nasz system dodawania komentarzy jest podatny na atak XSS.
- Aby temu zapobiec, należy dodać walidację symboli w formularzu, aby zabezpieczyć się przed przesłaniem np. skryptu JavaScript.
- Użyjmy funkcji `htmlspecialchars()` podczas pobierania danych od użytkownika oraz wyświetlania danych z bazy, aby mieć pewność, dodatkową że nie dojdzie do ataku.

<https://localhost/lecture13/postPagination.php>

Dziękuję za uwagę !

Bartosz Lauks